Solution: Single Number

Problem Statement

In a non-empty array of integers, every number appears twice except for one, find that single number.

Example 1:

```
Input: 1, 4, 2, 1, 3, 2, 3
Output: 4
```

Example 2:

```
Input: 7, 9, 7
Output: 9
```

Solution

One straight forward solution can be to use a **HashMap** kind of data structure and iterate through the input:

1. If number is already present in **HashMap**, remove it.
2. If number is not present in **HashMap**, add it.
3. In the end, only number left in the **HashMap** is our required single number.

Time and space complexity Time Complexity of the above solution will be O(n) and space complexity will also be O(n).

Can we do better than this using the **XOR Pattern**?

Solution with XOR

Recall the following two properties of XOR:

1. It returns zero if we take XOR of two same numbers.
2. It returns the same number if we XOR with zero.

So we can XOR all the numbers in the input; duplicate numbers will zero out each other and we will be left with the single number.

Code

Here is what our algorithm will look like:

Java

```java
class Solution {
  public int findSingleNumber(int[] arr) {
    int num = 0; // Initialize a variable to store the result
    for (int i = 0; i < arr.length; i++) {
      num = num ^ arr[i]; // Use XOR operation to find the single number
    }
    return num; // Return the found single number
  }

  public static void main(String args[]) {
    Solution sol = new Solution();
    System.out.println(sol.findSingleNumber(new int[]{1, 4, 2, 1, 3, 2, 3})); // Call th
  }
}
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

Time complexity of this solution is O(n) as we iterate through all numbers of the input once.

Space Complexity

The algorithm runs in constant space O(1).